

MANUAL BOOK

Sentiment Analysis on Universities under APERTI BUMN via Google Review

Abstract

Meningkatnya volume ulasan daring terhadap institusi pendidikan tinggi mendorong perlunya analisis sentimen otomatis untuk menangkap persepsi publik secara efisien. Penelitian ini mengembangkan model klasifikasi sentimen terhadap ulasan kampus APERTI BUMN dengan menerapkan tiga algoritma: Support Vector Machine (SVM), Random Forest, dan Regresi Logistik. Evaluasi menunjukkan bahwa SVM dengan hyperparameter tuning mencapai akurasi tertinggi sebesar 97,58%, menjadikannya model paling optimal dalam studi ini.

Kelompok 5

Benony Gabriel | Muh. Zacky Said | Nizar Afham Aflaha | Muh. Zaky Tabrani

I. DESKRIPSI PROYEK

Proyek ini berfokus pada pembangunan sistem analisis sentimen terhadap ulasan pengguna dari Google Maps yang ditujukan kepada universitas-universitas yang tergabung dalam konsorsium APERTI BUMN (Aliansi Perguruan Tinggi Badan Usaha Milik Negara). Ulasan yang dianalisis berasal dari beberapa universitas diantaranya:

- Universitas Pertamina (UPer)
- BRI Institut
- Institut Teknologi PLN (ITPLN)
- Institut Teknologi Telkom Surabaya (ITTS)
- Telkom University (Tel-U)
- Universitas Internasional Semen Indonesia (UISI)
- Universitas Logistik dan Bisnis Internasional (ULBI)

Tujuan utama dari proyek ini adalah untuk mengklasifikasikan sentimen ulasan menjadi tiga kategori, yaitu positif, netral, dan negatif, menggunakan algoritma pembelajaran mesin (machine learning). Dengan sistem ini, diharapkan institusi terkait dapat lebih memahami persepsi publik terhadap layanan dan lingkungan kampus mereka secara otomatis dan objektif.

Dataset telah disediakan dalam bentuk CSV dan diperoleh melalui proses *web scraping* dari halaman review Google Maps. Data tersebut kemudian diolah melalui berbagai tahapan seperti preprocessing teks, konversi teks ke bentuk numerik menggunakan TF-IDF, serta penanganan ketidakseimbangan data menggunakan metode SMOTE. Model machine learning yang digunakan dalam proyek ini meliputi:

- **Support Vector Machine (SVM)** sebagai model utama,
- **Logistic Regression** sebagai baseline sederhana, dan
- **Random Forest** sebagai model ensemble.

Model terbaik dipilih berdasarkan metrik evaluasi seperti akurasi, precision, recall, dan f1-score. Model yang telah dilatih disimpan agar dapat digunakan kembali untuk prediksi tanpa perlu melatih ulang. Proyek ini ditujukan untuk mendukung pengambilan keputusan berbasis data secara efisien dan akurat dalam konteks evaluasi layanan pendidikan tinggi.

II. SPESIFIKASI SISTEM

Komponen	Rincian
Bahasa Pemrograman	Python 3.10
Library Utama	Pandas, Numpy, scikit-learn,
Model	SVM, Random Forest, Logistic Regression
Editor/Kode IDE	VS Code / Jupyter Notebook
Lingkungan Virtual	Venv (direkomendasikan untuk isolasi paket)

III. PERSIAPAN LINGKUNGAN KERJA

Bagian ini menjelaskan tahapan dan alat yang diperlukan untuk menyiapkan lingkungan kerja sebelum menjalankan proyek analisis sentimen menggunakan Python dan Machine Learning.

A. Instalasi Python

- Pastikan Python versi **3.10** atau lebih baru telah terinstal.
- Unduh dari: <https://www.python.org/downloads/>
- Untuk memastikan instalasi berhasil, jalankan perintah berikut:

```
python --version
```

B. Membuat Virtual Environment

Virtual environment digunakan agar dependensi proyek terisolasi dan tidak mengganggu sistem global.

- Jalankan perintah berikut di terminal untuk membuat venv:

```
python -m venv .env
```

- Aktifkan environment:

- Windows

```
.env\Scripts\activate
```

- Linux/macOS

```
source .env/bin/activate
```

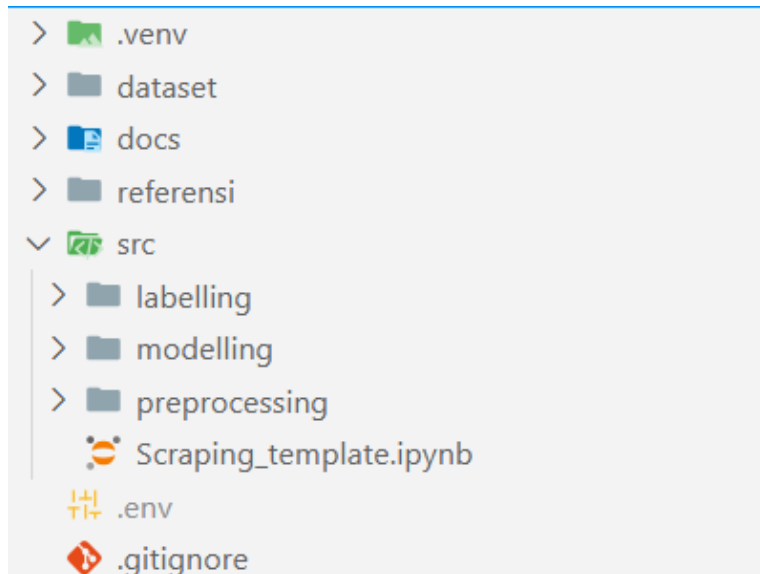
C. Instalasi Library dan Dependensi

Pastikan berada dalam environment yang sudah aktif, kemudian jalankan:

```
pip install pandas numpy scikit-learn matplotlib seaborn  
nltk Sastrawi imbalanced-learn
```

D. Struktur Folder Proyek

Berikut struktur direktori yang disarankan:



IV. MENU DAN CARA PENGGUNAAN

A. Labelling Data

Tujuan dari proses pelabelan dataset adalah untuk mengkategorikan ulasan (review) yang diperoleh dari Google Maps ke dalam tiga jenis sentimen utama, yaitu:

- Positif
- Negatif
- Netral

Label sentimen ini diperlukan sebagai target dalam pemodelan machine learning yang bertujuan untuk memahami persepsi pengguna terhadap institusi pendidikan (dalam hal ini, kampus di bawah naungan APERTI BUMN).

Proses labelling dilakukan menggunakan model GPT-3.5 Turbo dari OpenAI melalui API sebagai mesin klasifikasi sentimen karena beberapa alasan utama berikut:

- **Kemampuan Bahasa Alami:** GPT-3.5 Turbo sangat andal dalam memahami konteks bahasa Indonesia, termasuk ekspresi-emosi dalam teks informal seperti review pengguna.
- **Tidak Memerlukan Data Latih:** Berbeda dengan metode supervised learning konvensional yang memerlukan data berlabel terlebih dahulu, GPT-3.5 dapat digunakan langsung (*zero-shot classification*) untuk memberi label pada teks.
- **Konsistensi & Efisiensi Waktu:** Pelabelan manual oleh manusia rawan subjektivitas dan sangat memakan waktu. Dengan API GPT, proses dapat diotomatisasi secara akurat dan konsisten.
- **Fleksibel:** Bisa digunakan untuk berbagai domain dan jenis teks dengan sedikit atau tanpa penyesuaian.

1. Daftar ke openai platform untuk mendapatkan API Key

- Website resmi: <https://openai.com/api/>

2. Import Library

```
from openai import OpenAI
import pandas as pd
import time
import os
from dotenv import load_dotenv
```

3. Load API Key dan Baca Dataset

```
# Load API Key dari .env
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# === Baca file CSV ===
df_ItBRI = pd.read_csv('../dataset/data_ItBRI.csv')
df_ItPLN = pd.read_csv('../dataset/data_ItPLN.csv')
df_ITTS = pd.read_csv('../dataset/data_ITTS.csv')
df_Tel_U = pd.read_csv('../dataset/data_Tel-U.csv')
df_UISI = pd.read_csv('../dataset/data_UISI.csv')
df_ULBI = pd.read_csv('../dataset/data_ULBI.csv')
df_Uper = pd.read_csv('../dataset/data_Uper.csv')
```

4. Cek jumlah baris dan kolom dataset

```
# Cek jumlah baris dan kolom
print("Data ItBRI: " + str(df_ItBRI.shape[0]) + " rows, " +
      str(df_ItBRI.shape[1]) + " columns")
print("Data ItPLN: " + str(df_ItPLN.shape[0]) + " rows, " +
      str(df_ItPLN.shape[1]) + " columns")
print("Data ITTS: " + str(df_ITTS.shape[0]) + " rows, " +
      str(df_ITTS.shape[1]) + " columns")
print("Data Tel-U: " + str(df_Tel_U.shape[0]) + " rows, " +
      str(df_Tel_U.shape[1]) + " columns")
print("Data UISI: " + str(df_UISI.shape[0]) + " rows, " +
      str(df_UISI.shape[1]) + " columns")
print("Data ULBI: " + str(df_ULBI.shape[0]) + " rows, " +
      str(df_ULBI.shape[1]) + " columns")
print("Data Uper: " + str(df_Uper.shape[0]) + " rows, " +
      str(df_Uper.shape[1]) + " columns")
```

Output:

```
Data ItBRI: 78 rows, 9 columns
Data ItPLN: 188 rows, 9 columns
Data ITTS: 308 rows, 9 columns
Data Tel-U: 988 rows, 9 columns
Data UISI: 188 rows, 9 columns
Data ULBI: 596 rows, 9 columns
Data Uper: 803 rows, 9 columns
```

5. Bangun fungsi klasifikasi sentimen untuk pelabelan

```
# === Load data CSV ===
df_Tel_U = pd.read_csv('../dataset/data_Tel-U.csv')
df_Tel_U['label_sentimen'] = None # Tambahkan kolom baru

# === Fungsi klasifikasi sentimen ===
def label_sentimen(snippet):
    try:
        prompt = f"""
            Tolong klasifikasikan sentimen dari
            review berikut ke dalam salah satu dari tiga label: positif,
            negatif, atau netral.

            Review: "{snippet}"

            Jawaban hanya satu kata: positif /
            negatif / netral.
            """
        response = client.chat.completions.create(
            model="gpt-3.5-turbo", # atau gpt-4
            messages=[
                {"role": "system", "content": "Kamu adalah
            asisten untuk klasifikasi sentimen review kampus."},
                {"role": "user", "content": prompt}
            ],
            temperature=0.2,
        )
        label =
        response.choices[0].message.content.strip().lower()

        if label in ['positif', 'negatif', 'netral']:
            return label
        else:
            return 'netral'
    except Exception as e:
        print(f"Terjadi error: {e}")
        return 'netral'
```

Prompt tersebut dikirim ke API menggunakan endpoint **chat.completions.create()** dari library openai versi terbaru (≥ 1.0). Response dari model akan berupa satu kata hasil klasifikasi (positif/negatif/netral).

6. Lakukan loop untuk melabeli setiap review

```
# === Loop untuk melabeli setiap review ===
for i, row in df_Tel_U.iterrows():
    if pd.notnull(row['snippet']):
```

```
print(f"Labeling review ke-{i+1}...")
df_Tel_U.at[i, 'label_sentimen'] =
label_sentimen(row['snippet'])
time.sleep(1.5) # Hindari rate limit
```

7. Validasi label

Hasil respon dari API divalidasi agar sesuai dengan tiga label yang ditentukan. Validasi dilakukan secara manual terhadap Setelah itu, label disimpan kembali ke dataset pada kolom baru `label_sentimen`.

8. Simpan dataset

```
# Simpan ke file baru
output_dir = '../..dataset/labeled_data/'
os.makedirs(output_dir, exist_ok=True)
df_Tel_U.to_csv(os.path.join(output_dir, 'Tel-
U_labelled.csv'), index=False)
print("Labeling selesai. File hasil disimpan.")
```

9. Setelah semua snippet diberi label, dataset akan memiliki kolom baru **label_sentimen** yang siap digunakan untuk tahap selanjutnya. Hasil labelling akan tersimpan dalam folder **dataset/labeled_data**.

B. Preprocessing dan Exploratory Data Analysis

Tahapan *preprocessing* bertujuan untuk membersihkan dan menyiapkan data agar dapat digunakan secara optimal dalam pelatihan model. Langkah-langkah preprocessing dalam proyek ini meliputi:

- Menghapus kolom yang tidak diperlukan
- Lowercasing
- Handling missing values
- Menghapus whitespace
- Hapus punctuation
- Normalisasi
- Tokenisasi
- Handling imbalance data

Pada tahapan ini dilakukan preprocessing dan EDA untuk tiap dataset. Pada tahapan ini, kami menggunakan dataset Tel-U sebagai contoh untuk menampilkan langkah-langkah yang dilakukan. Silahkan lakukan langkah-langkah yang sama untuk dataset yang lain.

1. Preprocessing & Cleaning

1) Import Library

```
import pandas as pd
import string
import re
import ast
import numpy as np
```

```
import nltk
import seaborn as sns
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
```

2) Load dataset dan lihat struktur data

```
df = pd.read_csv('../dataset/labeled_data/Tel-
U_labelled.csv')

print(df.shape)
print(df.info())
df.head()
```

3) Lihat sebaran rating

```
df['rating'].describe()
```

count	988.000000
mean	4.411943
std	1.186439
min	1.000000
25%	4.000000
50%	5.000000
75%	5.000000
max	5.000000
Name: rating, dtype: float64	

4) Hapus kolom yang tidak diperlukan

Beberapa kolom pada dataset yang tidak memiliki kontribusi terhadap analisis sentimen dihapus, seperti 'page', 'name', 'link', 'thumbnail', 'date', 'images', dan 'local_guide'. Langkah ini bertujuan untuk menyederhanakan data dan hanya mempertahankan fitur yang relevan, yaitu kolom 'snippet' (teks ulasan) dan label sentimen.

```
df.columns
# drop the columns
df = df.drop(columns=['page', 'name', 'link',
'thumbnail', 'date', 'images', 'local_guide'], axis=1)
df.head(10)

Index(['page', 'name', 'link', 'thumbnail', 'rating', 'date', 'snippet',
'images', 'local_guide', 'label_sentimen'],
      dtype='object')
```

	rating	snippet	label_sentimen
0	4.0	Kampus swasta di area yg padat, luas sekali t...	negatif
1	1.0	Siang hari ini hadir wisuda anak. Tempat wisud...	negatif
2	5.0	jangan lupa follow lg : mfft_27\ntnrmksh 🙌🥳	netral
3	5.0	Kampus swasta ternama, lokasi di bojongsong k...	positif
4	1.0	Yang terhormat bapak kepala satpam dan manajem...	negatif
5	3.0	Securitynya disana engga ramah ya, terus pada ...	negatif
6	5.0	2016, Bisa Bikin Brand, Terimakasih Ilmunya Pa...	positif
7	1.0	ATURAN SIH ATURAN MASA DRIVER ONLINE OJOL MASU...	negatif
8	1.0	WASTE TIME BANGET DI SURUH JALAN KE ASRAMA PUT...	negatif
9	5.0	Kampus swasta terbaik di Indonesia. Kereeen...	positif

5) Convert to lowercase

Seluruh teks dikonversi menjadi huruf kecil (lowercase) untuk memastikan konsistensi dan mencegah penggandaan kata akibat perbedaan kapitalisasi. Contoh: "Bagus" dan "bagus" dianggap sebagai kata yang sama.

```
df['snippet'] = df['snippet'].str.lower()
df.head()
```

	rating	snippet	label_sentimen
0	4.0	kampus swasta di area yg padat, luas sekali t...	negatif
1	1.0	siang hari ini hadir wisuda anak. tempat wisud...	negatif
2	5.0	jangan lupa follow ig : mfft_27\ntnrmksh 🙌🥳	netral
3	5.0	kampus swasta ternama, lokasi di bojongsong k...	positif
4	1.0	yang terhormat bapak kepala satpam dan manajem...	negatif

6) Hapus missing values

Semua baris yang memiliki nilai kosong (missing/null) pada kolom 'snippet' dari dataset. Hal ini dilakukan agar tidak ada data tidak valid yang masuk ke tahap pelatihan model.

```
df.isnull().sum()
df = df.dropna()
df.head()
df.isnull().sum()
```

7) Menghapus whitespace / spasi berlebihan

Spasi ganda, tab, dan karakter whitespace lain yang tidak diperlukan dihapus atau diganti dengan satu spasi. Contoh: "ini bagus" → "ini bagus".

```
def remove_extra_whitespace(text):
    # Menghapus spasi di awal dan akhir teks
    text = text.strip()
    # Menghapus spasi berlebih di tengah teks
```

```

text = re.sub('\s+', ' ', text)
return text

df['snippet'] =
df['snippet'].apply(remove_extra_whitespace)

```

8) Hapus punctuation

Seluruh tanda baca seperti titik (.), koma (,), tanda seru (!), dan lainnya dihapus karena tidak memberikan kontribusi signifikan dalam analisis sentimen berbasis kata.

```

string.punctuation
def remove_punctuations(text):
    punctuations = string.punctuation
    return text.translate(str.maketrans('', '',
punctuations))

df['snippet'] = df['snippet'].apply(remove_punctuations)
df.head()

```

9) Visualisasi sebelum preprocessing

```

# Gabungkan semua teks dari kolom 'snippet' yang tidak
null
text = " ".join(df['snippet'].dropna())

# Buat dan tampilkan wordcloud
wordcloud = WordCloud(width=800, height=400,
background_color='white').generate(text)
plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Wordcloud Kolom Snippet')
plt.show()

```

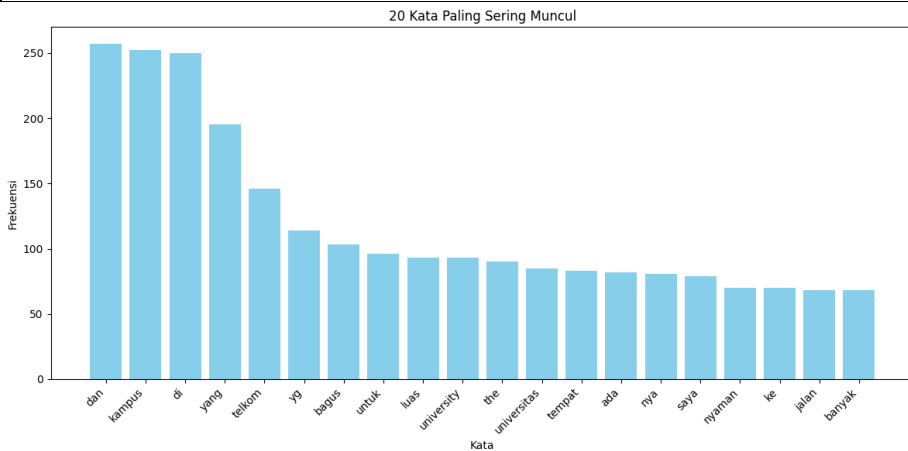


```
from collections import Counter

# Tokenisasi dan hitung frekuensi kata
words = text.split()
word_freq = Counter(words)

# Ambil 20 kata paling sering muncul
most_common = word_freq.most_common(20)
words_plot, counts_plot = zip(*most_common)

plt.figure(figsize=(12,6))
plt.bar(words_plot, counts_plot, color='skyblue')
plt.xticks(rotation=45, ha='right')
plt.title('20 Kata Paling Sering Muncul')
plt.xlabel('Kata')
plt.ylabel('Frekuensi')
plt.tight_layout()
plt.show()
```



10) Normalisasi

Dilakukan normalisasi terhadap kata-kata tidak baku atau bahasa gaul ke dalam bentuk baku. Contoh: "gk" $\rightarrow$ "tidak", "bgt" $\rightarrow$ "banget", "mkn" $\rightarrow$ "makan". Langkah ini penting dalam konteks bahasa

Indonesia karena banyak pengguna menulis dengan gaya informal atau singkatan.

```
# Load kamus slang
with open('kamus_slang.json', encoding='utf-8') as f:
    slang_dict = ast.literal_eval(f.read())

# Fungsi untuk mengganti kata slang dengan kata yang
# sudah dinormalisasi
def normalize_text(text):
    words = text.split()
    normalized_words = [slang_dict.get(word, word) for
word in words]
    return " ".join(normalized_words)

# Menerapkan normalisasi pada kolom snippet
df['snippet'] = df['snippet'].apply(lambda x:
normalize_text(x))
df.head(10)
```

11) Tokenisasi

Kalimat dipecah menjadi daftar kata (token) untuk mempermudah proses analisis dan transformasi teks ke dalam bentuk numerik.

```
def tokenize_text(text):
    # Tokenisasi teks
    tokens = text.split()
    # Mengembalikan daftar token
    return tokens

df['tokens'] = df['snippet'].apply(tokenize_text)

df.head(10)
```

12) Visualisasi setelah preprocessing

```
# Gabungkan semua token dari kolom 'tokens' yang tidak
# null menjadi satu string
all_tokens = [token for tokens in df['tokens'].dropna()
for token in tokens]
text = " ".join(all_tokens)

# Buat dan tampilkan wordcloud
wordcloud = WordCloud(width=800, height=400,
background_color='white').generate(text)
plt.figure(figsize=(12, 6))
```

```
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Wordcloud Kolom tokens')
plt.show()
```

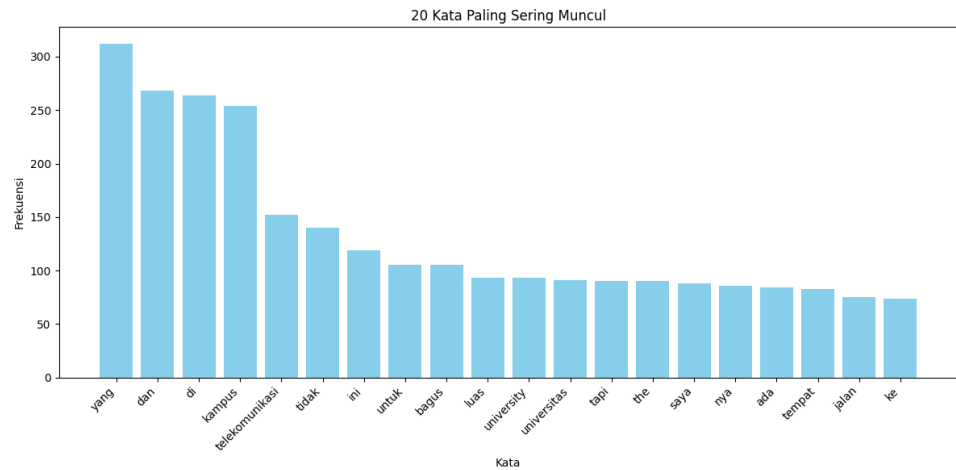


```
from collections import Counter

# Tokenisasi dan hitung frekuensi kata
words = text.split()
word_freq = Counter(words)

# Ambil 20 kata paling sering muncul
most_common = word_freq.most_common(20)
words_plot, counts_plot = zip(*most_common)

plt.figure(figsize=(12,6))
plt.bar(words_plot, counts_plot, color='skyblue')
plt.xticks(rotation=45, ha='right')
plt.title('20 Kata Paling Sering Muncul')
plt.xlabel('Kata')
plt.ylabel('Frekuensi')
plt.tight_layout()
plt.show()
```



13) Simpan dataset bersih

```
# Ubah urutan kolom sebelum menyimpan
df = df[['rating', 'snippet', 'tokens',
        'label_sentimen']]

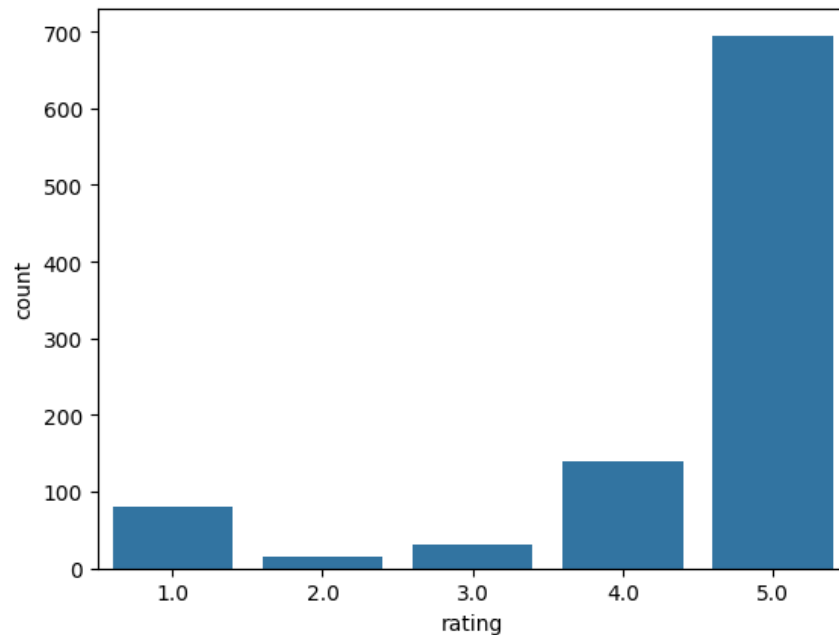
output_path = '../..dataset/cleaned_dataset/Tel-
U_labelled_cleaned.csv'

try:
    # Save the dataframe to a CSV file
    df.to_csv(output_path, index=False)
    print(f"DataFrame successfully saved to
    {output_path}")
except Exception as e:
    print(f"An error occurred: {e}")
```

2. Exploratory Data Analysis

1) Visualisasi rating

```
df['rating'].unique()
sns.countplot(x=df['rating'])
plt.show()
print(df['rating'].value_counts())
```



2) Visualisasi label

```
# Visualisasi Wordcloud untuk masing-masing sentimen pada
# kolom tokens
sentiments = ['positif', 'negatif', 'netral']
plt.figure(figsize=(18, 5))

for i, sentiment in enumerate(sentiments):
    # Gabungkan semua token menjadi satu string untuk
    # sentimen tertentu
    tokens_sentiment = df[df['label_sentimen'] ==
sentiment]['tokens'].dropna()
    text_sentiment = " ".join([token for tokens in
tokens_sentiment for token in tokens])
    wordcloud = WordCloud(width=600, height=300,
background_color='white').generate(text_sentiment)
    plt.subplot(1, 3, i+1)
    plt.imshow(wordcloud, interpolation='bilinear')
    plt.axis('off')
    plt.title(f'Wordcloud Sentimen
{sentiment.capitalize()}')

plt.tight_layout()
plt.show()
```



3) Visualisasi distribusi jumlah karakter dalam ulasan per label

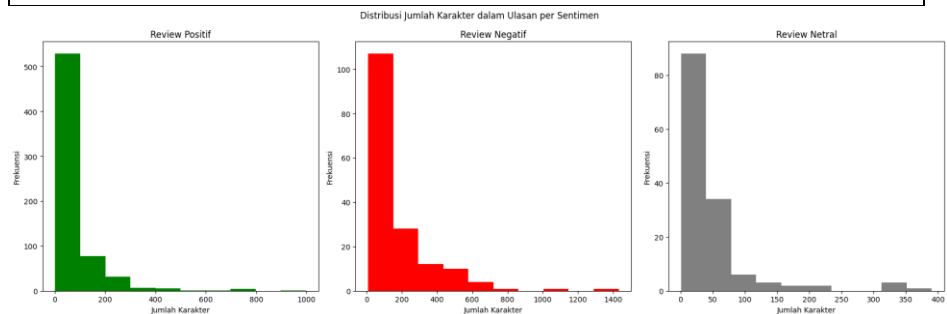
```

figure, axes = plt.subplots(1, 3, figsize=(18, 6))
sentiments = ['positif', 'negatif', 'netral']
colors = ['green', 'red', 'gray']

for i, sentiment in enumerate(sentiments):
    review_lengths = df[df['label_sentimen'] ==
sentiment]['snippet'].str.len()
    axes[i].hist(review_lengths, color=colors[i])
    axes[i].set_title(f'Review {sentiment.capitalize()}')
    axes[i].set_xlabel('Jumlah Karakter')
    axes[i].set_ylabel('Frekuensi')

figure.suptitle('Distribusi Jumlah Karakter dalam Ulasan
per Sentimen')
plt.tight_layout()
plt.show()

```



4) Visualisasi panjang kata per sentimen

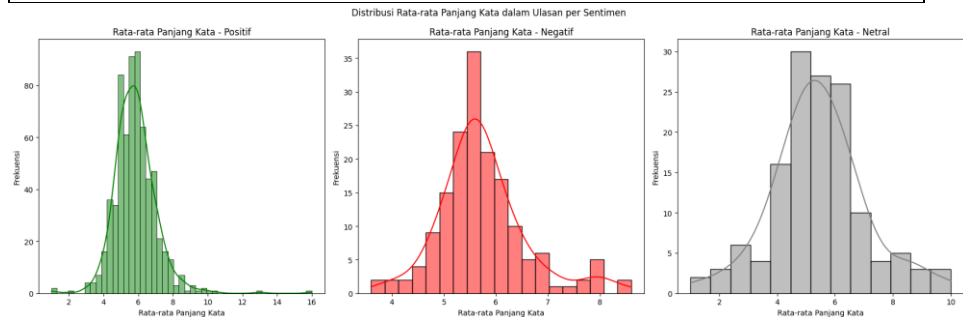
```

figure, axes = plt.subplots(1, 3, figsize=(18, 6))
sentiments = ['positif', 'negatif', 'netral']
colors = ['green', 'red', 'gray']

for i, sentiment in enumerate(sentiments):
    # Ambil token untuk setiap sentimen
    tokens = df[df['label_sentimen'] ==
sentiment]['tokens']
    # Hitung rata-rata panjang kata per ulasan
    avg_word_length = tokens.apply(lambda x:
np.mean([len(word) for word in x]) if x else 0)
    sns.histplot(avg_word_length, ax=axes[i],
color=colors[i], kde=True)
    axes[i].set_title(f'Rata-rata Panjang Kata -
{sentiment.capitalize()}')
    axes[i].set_xlabel('Rata-rata Panjang Kata')
    axes[i].set_ylabel('Frekuensi')

```

```
figure.suptitle('Distribusi Rata-rata Panjang Kata dalam
Ulasan per Sentimen')
plt.tight_layout()
plt.show()
```



5) Visualisasi kata-kata dalam sentimen

```
from collections import Counter
import plotly.express as px

def get_top_token_ngrams(token_series, n=20, ngram=1):
    ngram_list = []
    for tokens in token_series.dropna():
        ngram_list.extend([' '.join(tokens[i:i+ngram])
        for i in range(len(tokens)-ngram+1)])
    return Counter(ngram_list).most_common(n)

# Visualisasi untuk sentimen positif
tokens_positif = df[df['label_sentimen'] ==
'positif']['tokens']
most_common_uni_positif =
get_top_token_ngrams(tokens_positif, 20, 1)
most_common_uni_positif = dict(most_common_uni_positif)

temp_positif = pd.DataFrame({
    "Common_words": list(most_common_uni_positif.keys()),
    "Count": list(most_common_uni_positif.values())
})

fig_positif = px.bar(
    temp_positif, x="Count", y="Common_words",
    title="Common Words in Positif Reviews (Tokens)",
    orientation='h', width=700, height=700,
    color="Common_words"
)
fig_positif.show()

# Visualisasi untuk sentimen negatif
```

```

tokens_negatif = df[df['label_sentimen'] ==
'negatif']['tokens']
most_common_uni_negatif =
get_top_token_ngrams(tokens_negatif, 20, 1)
most_common_uni_negatif = dict(most_common_uni_negatif)

temp_negatif = pd.DataFrame({
    "Common_words": list(most_common_uni_negatif.keys()),
    "Count": list(most_common_uni_negatif.values())
})

fig_negatif = px.bar(
    temp_negatif, x="Count", y="Common_words",
    title="Common Words in Negatif Reviews (Tokens)",
    orientation='h', width=700, height=700,
    color="Common_words"
)
fig_negatif.show()

# Visualisasi untuk sentimen netral
tokens_netral = df[df['label_sentimen'] ==
'netral']['tokens']
most_common_uni_netral =
get_top_token_ngrams(tokens_netral, 20, 1)
most_common_uni_netral = dict(most_common_uni_netral)

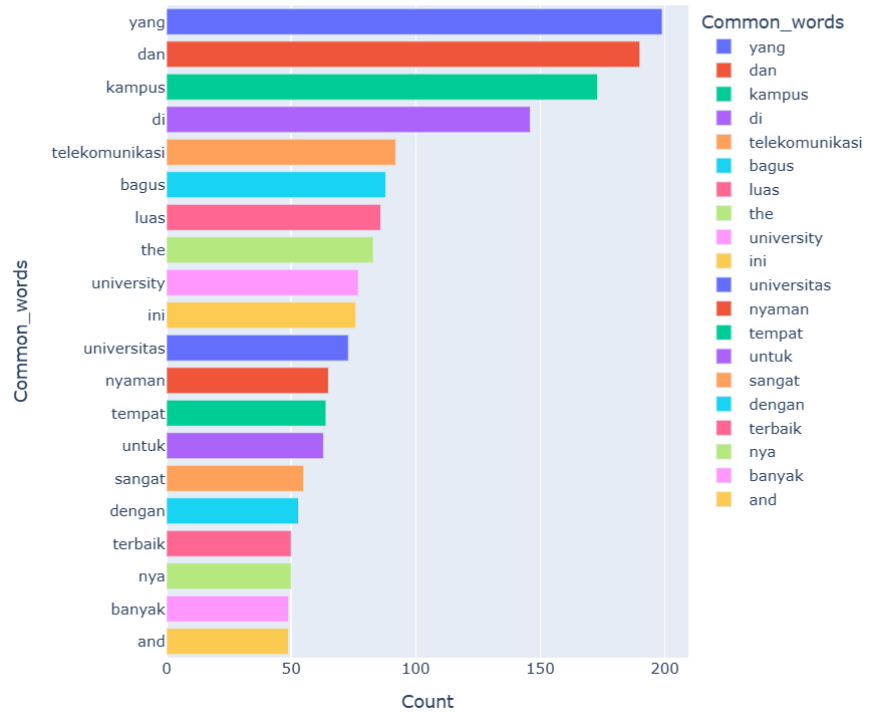
temp_netral = pd.DataFrame({
    "Common_words": list(most_common_uni_netral.keys()),
    "Count": list(most_common_uni_netral.values())
})

fig_netral = px.bar(
    temp_netral, x="Count", y="Common_words",
    title="Common Words in Netral Reviews (Tokens)",
    orientation='h', width=700, height=700,
    color="Common_words"
)
fig_netral.show()

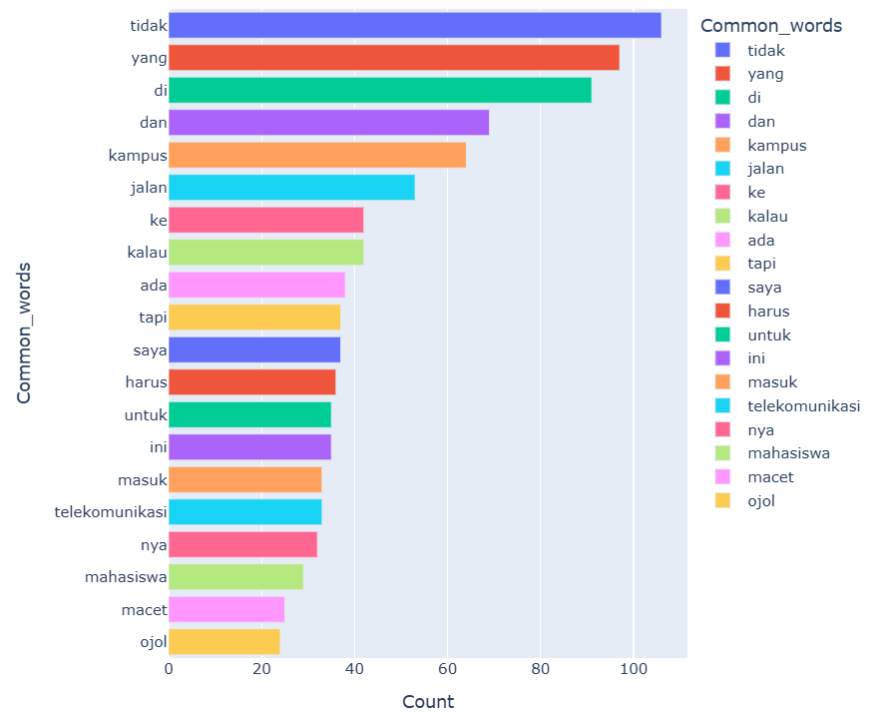
```

MANUAL BOOK

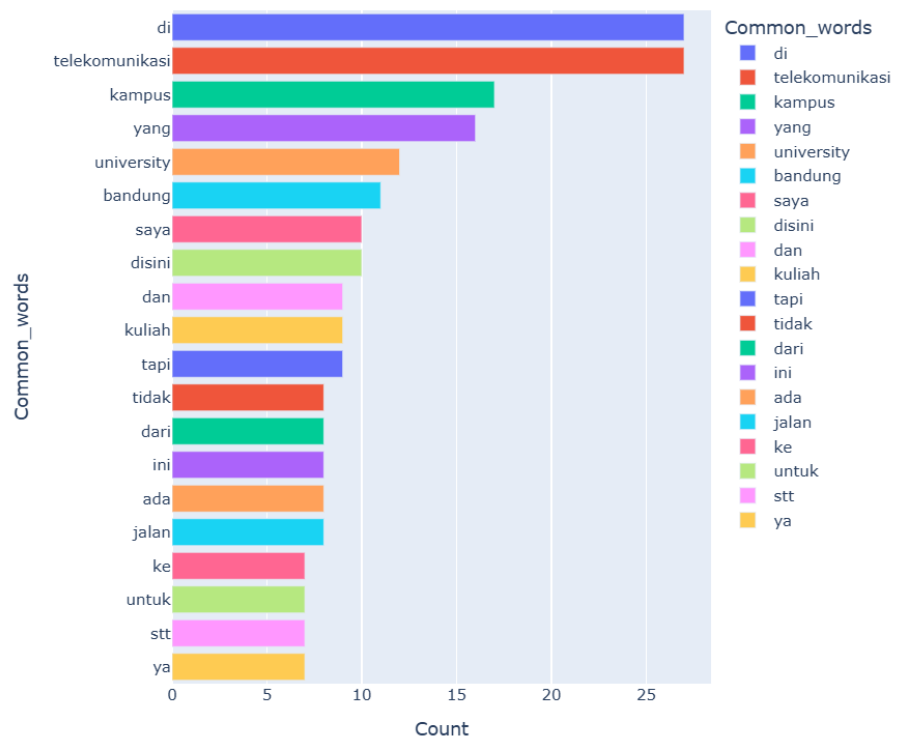
Common Words in Positif Reviews (Tokens)



Common Words in Negatif Reviews (Tokens)



Common Words in Netral Reviews (Tokens)



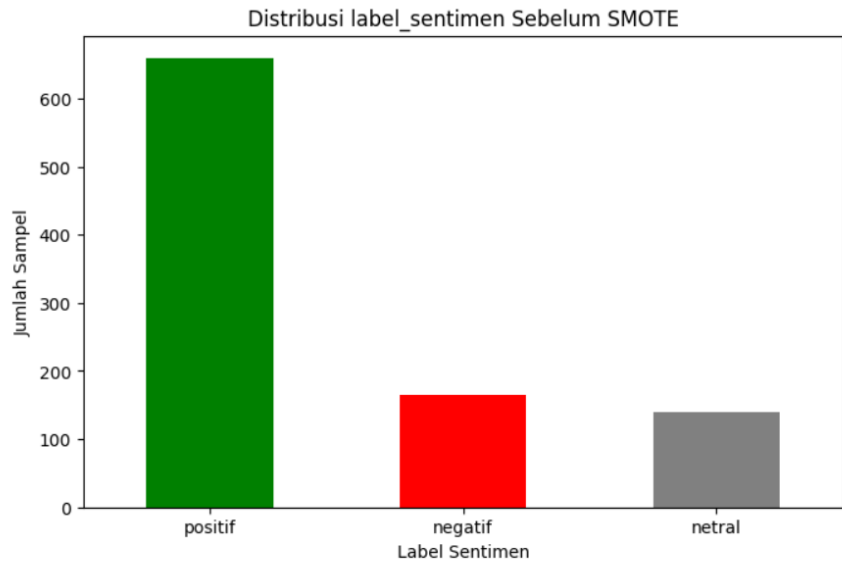
3. Handling Imbalance data

Disitribusi kelas tidak seimbang (imbalance data) pada kelas positif, negatif, dan netral sehingga perlu dilakukan balancing data. Hal ini dapat dilakukan dengan teknik *oversampling* atau *undersampling*. Disini kami melakukannya dengan teknik *oversampling* yaitu dengan menggunakan metode SMOTE.

- Visualisasi distribusi kelas sebelum SMOTE

```
# Visualisasi distribusi label_sentimen sebelum SMOTE
plt.figure(figsize=(8, 5))
y.value_counts().plot(kind='bar', color=['green',
'green', 'red', 'gray'])
plt.title('Distribusi label_sentimen Sebelum SMOTE')
plt.xlabel('Label Sentimen')
plt.ylabel('Jumlah Sampel')
plt.xticks(rotation=0)
plt.show()

y.value_counts()
```



```
label_sentimen
positif    659
negatif    164
netral     139
Name: count, dtype: int64
```

- Import library

```
from sklearn.feature_extraction.text import
TfidfVectorizer
from imblearn.over_sampling import SMOTE
from collections import Counter
```

- Lakukan TF-IDF

```
# TF-IDF Vectorization
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['text'])

# Label
y = df['label_sentimen']

# Sebelum SMOTE
print("Distribusi label sebelum SMOTE:", Counter(y))
```

- Terapkan SMOTE

```
# Terapkan SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Setelah SMOTE
```

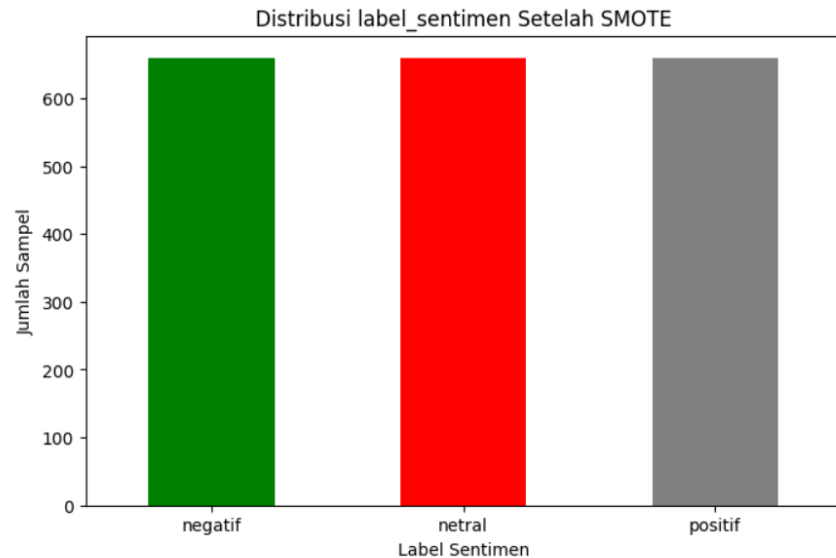
```
print("Distribusi label setelah SMOTE:",  
Counter(y_resampled))
```

- Simpan dataframe

```
# Simpan hasilnya dalam DataFrame  
X_res_df = pd.DataFrame(X_resampled.toarray(),  
columns=vectorizer.get_feature_names_out())  
y_res_df = pd.DataFrame(y_resampled,  
columns=["label_sentimen"])  
  
# Gabungkan hasil  
df = pd.concat([X_res_df, y_res_df], axis=1)
```

- Visualisasi label setelah SMOTE

```
import matplotlib.pyplot as plt  
  
# Visualisasi distribusi label_sentimen setelah SMOTE  
plt.figure(figsize=(8, 5))  
y_res_df['label_sentimen'].value_counts().plot(kind='bar', color=['green', 'red', 'gray'])  
plt.title('Distribusi label_sentimen Setelah SMOTE')  
plt.xlabel('Label Sentimen')  
plt.ylabel('Jumlah Sampel')  
plt.xticks(rotation=0)  
plt.show()  
  
df['label_sentimen'].value_counts()
```



```
label_sentimen
negatif    659
netral     659
positif    659
Name: count, dtype: int64
```

- Simpan dataset balanced

```
output_path = '../..../dataset/balanced_data/Tel-
U_labelled_balanced.csv'

try:
    # Save the dataframe to a CSV file
    df.to_csv(output_path, index=False)
    print(f"DataFrame successfully saved to
{output_path}")
except Exception as e:
    print(f"An error occurred: {e}")
```

C. Modelling

Pada proyek ini, tiga algoritma supervised learning digunakan untuk membangun model klasifikasi sentimen, yaitu:

- Support Vector Machine (SVM)
- Random Forest
- Logistic Regression

Ketiga model ini dipilih karena telah terbukti efektif dan banyak digunakan dalam penelitian NLP. Setiap model dilatih menggunakan fitur hasil ekstraksi TF-IDF dari teks yang telah melalui tahapan preprocessing. Evaluasi dilakukan menggunakan metrik akurasi dan classification report (precision, recall, F1-score) pada data uji.

1. Import library dan load dataset

```
import pandas as pd
from sklearn.model_selection import train_test_split,
GridSearchCV
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,
accuracy_score

df = pd.read_csv('all_campus_combined.csv')
```

2. Lihat data awal

```
df['kampus'].unique()
df = df.drop(columns=['kampus'])
```

3. Split data menjadi train dan test (80:20)

```
X = df.drop('label_sentimen', axis=1)
y = df['label_sentimen'] # target

# === Split data menjadi train dan test set ===
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
```

4. SVM Model

SVM dikenal sangat efektif dalam ruang berdimensi tinggi seperti representasi teks hasil ekstraksi TF-IDF. SVM bekerja dengan mencari hyperplane optimal yang memisahkan kelas-kelas dalam data. Keunggulan SVM terletak pada kemampuannya menangani kasus klasifikasi dengan margin sempit dan data yang tidak selalu terdistribusi secara linier.

```
from sklearn.impute import SimpleImputer

# Impute missing values with 0
imputer = SimpleImputer(strategy='constant', fill_value=0)
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# === Model SVM ===
print("\n=== Support Vector Machine (SVM) ===")
svm_model = SVC(kernel='linear', C=1, random_state=42)
svm_model.fit(X_train_imputed, y_train)
y_pred_svm = svm_model.predict(X_test_imputed)
print(classification_report(y_test, y_pred_svm))
```

```
print("Akurasi:", accuracy_score(y_test, y_pred_svm))
```

```
=== Support Vector Machine (SVM) ===
```

	precision	recall	f1-score	support
negatif	0.97	0.99	0.98	290
netral	0.82	0.98	0.90	290
positif	0.98	0.77	0.86	290
accuracy			0.91	870
macro avg	0.92	0.91	0.91	870
weighted avg	0.92	0.91	0.91	870

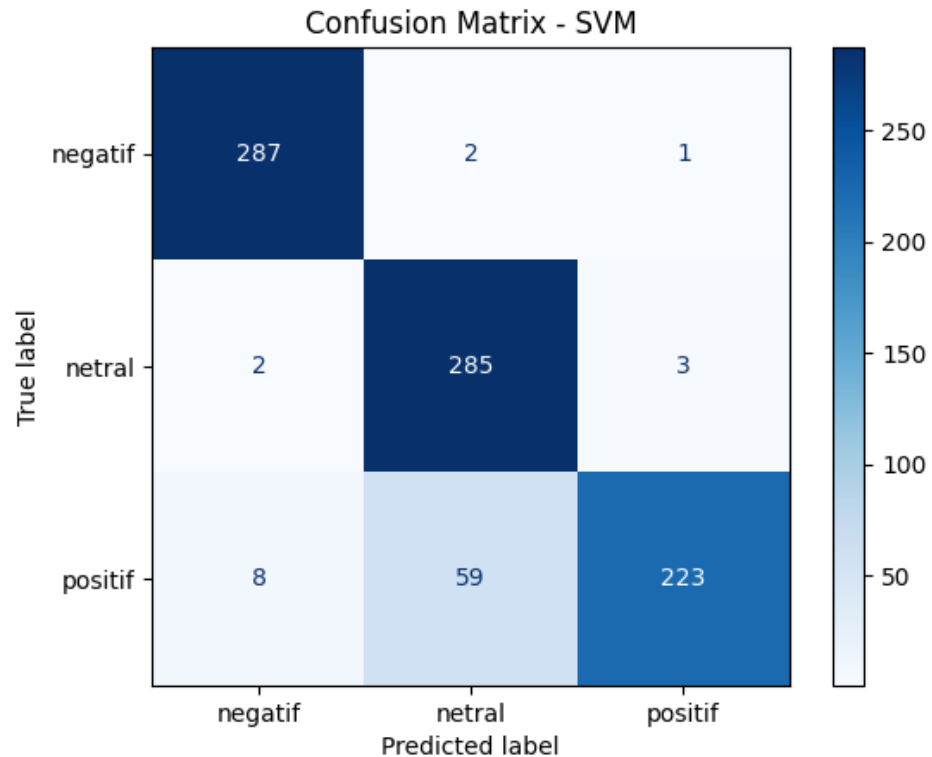
```
Akurasi: 0.9137931034482759
```

Visualisasi confusion matrix:

```
from sklearn.metrics import ConfusionMatrixDisplay,
confusion_matrix

import matplotlib.pyplot as plt

# Confusion matrix untuk model SVM
cm_svm = confusion_matrix(y_test, y_pred_svm,
labels=svm_model.classes_)
disp_svm = ConfusionMatrixDisplay(confusion_matrix=cm_svm,
display_labels=svm_model.classes_)
plt.figure(figsize=(5,4))
disp_svm.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - SVM")
plt.show()
```



Lakukan hyperparameter tuning dengan grid search cv:

```
# Tuning SVM
svm_params = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

# Impute missing values in X_train and X_test
X_train_imputed = imputer.transform(X_train)
X_test_imputed = imputer.transform(X_test)

svm_grid = GridSearchCV(SVC(), svm_params, cv=5,
                        scoring='accuracy')
svm_grid.fit(X_train_imputed, y_train)
svm_best = svm_grid.best_estimator_
svm_pred = svm_best.predict(X_test_imputed)
print("\n=== Support Vector Machine (SVM) ===")
print("Best Parameters:", svm_grid.best_params_)
print("Akurasi:", accuracy_score(y_test, svm_pred))
print(classification_report(y_test, svm_pred))
```

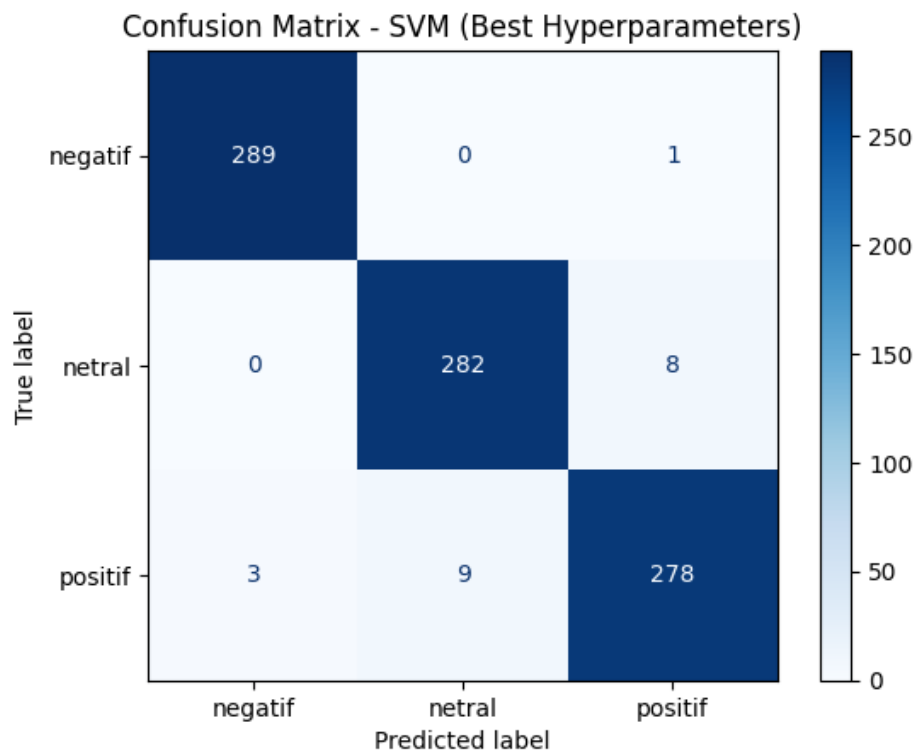
=== Support Vector Machine (SVM) ===

Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
 Akurasi: 0.9758620689655172

	precision	recall	f1-score	support
negatif	0.99	1.00	0.99	290
netral	0.97	0.97	0.97	290
positif	0.97	0.96	0.96	290
accuracy			0.98	870
macro avg	0.98	0.98	0.98	870
weighted avg	0.98	0.98	0.98	870

Visualisasi hasil hyperparameter tuning:

```
# Visualisasi confusion matrix untuk SVM hasil
hyperparameter tuning
cm_svm_best = confusion_matrix(y_test, svm_pred,
labels=svm_best.classes_)
disp_svm_best =
ConfusionMatrixDisplay(confusion_matrix=cm_svm_best,
display_labels=svm_best.classes_)
plt.figure(figsize=(5,4))
disp_svm_best.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - SVM (Best Hyperparameters)")
plt.show()
```



Visualisasi learning curve:

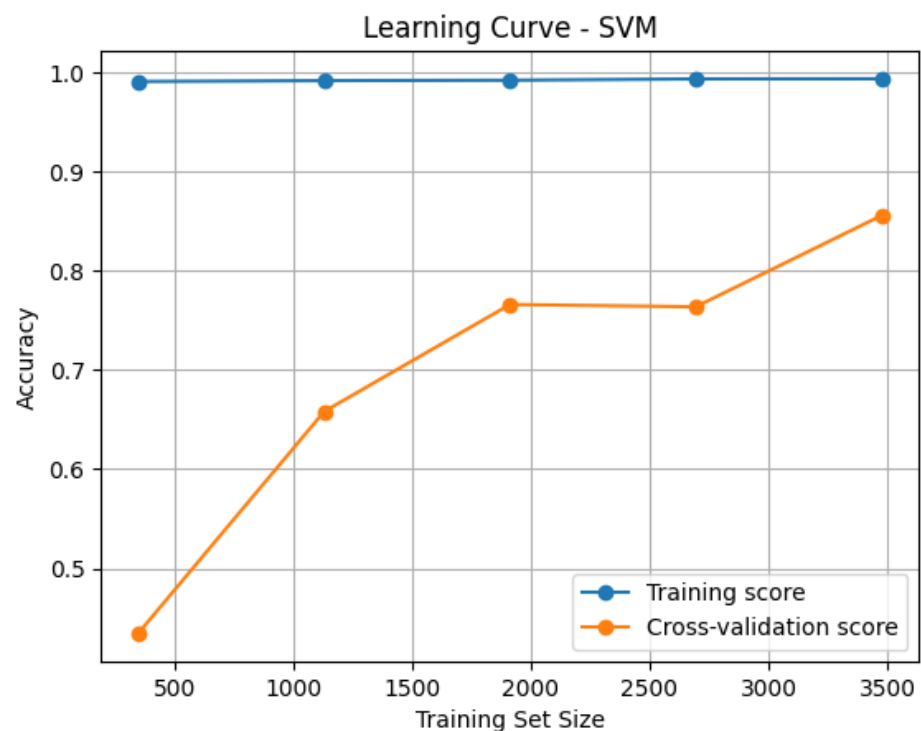
```
from sklearn.model_selection import learning_curve
import numpy as np

# Impute missing values in X before using learning_curve
X_imputed = imputer.transform(X)

train_sizes, train_scores, test_scores = learning_curve(
    svm_best, X_imputed, y, cv=5, scoring='accuracy',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 5)
)

train_scores_mean = train_scores.mean(axis=1)
test_scores_mean = test_scores.mean(axis=1)

plt.plot(train_sizes, train_scores_mean, 'o-',
label="Training score")
plt.plot(train_sizes, test_scores_mean, 'o-', label="Cross-
validation score")
plt.xlabel("Training Set Size")
plt.ylabel("Accuracy")
plt.title("Learning Curve - SVM")
plt.legend(loc="best")
plt.grid()
plt.show()
```



5. Random Forest Model

Random Forest adalah algoritma ensemble yang membangun banyak pohon keputusan dan menggabungkan prediksinya untuk meningkatkan akurasi dan mengurangi overfitting. Algoritma ini sangat andal ketika data memiliki noise dan tidak selalu linier. Random Forest juga memberikan feature importance, yang berguna untuk analisis interpretabilitas model.

```
print("\n=== Random Forest ===")
rf_model = RandomForestClassifier(n_estimators=100,
random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
print(classification_report(y_test, y_pred_rf))
print("Akurasi:", accuracy_score(y_test, y_pred_rf))
```

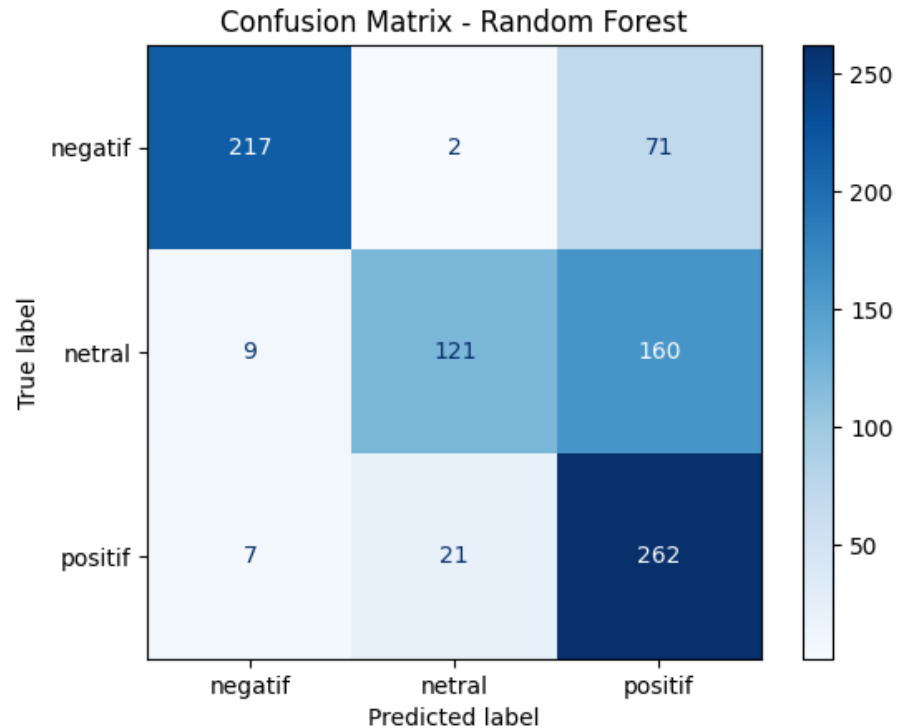
```
=== Random Forest ===
```

	precision	recall	f1-score	support
negatif	0.93	0.75	0.83	290
netral	0.84	0.42	0.56	290
positif	0.53	0.90	0.67	290
accuracy			0.69	870
macro avg	0.77	0.69	0.69	870
weighted avg	0.77	0.69	0.69	870

```
Akurasi: 0.6896551724137931
```

Visualisasi confusion matrix:

```
# Confusion matrix untuk model Random Forest
cm_rf = confusion_matrix(y_test, y_pred_rf,
labels=rf_model.classes_)
disp_rf = ConfusionMatrixDisplay(confusion_matrix=cm_rf,
display_labels=rf_model.classes_)
plt.figure(figsize=(5,4))
disp_rf.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - Random Forest")
plt.show()
```



Lakukan hyperparameter tuning:

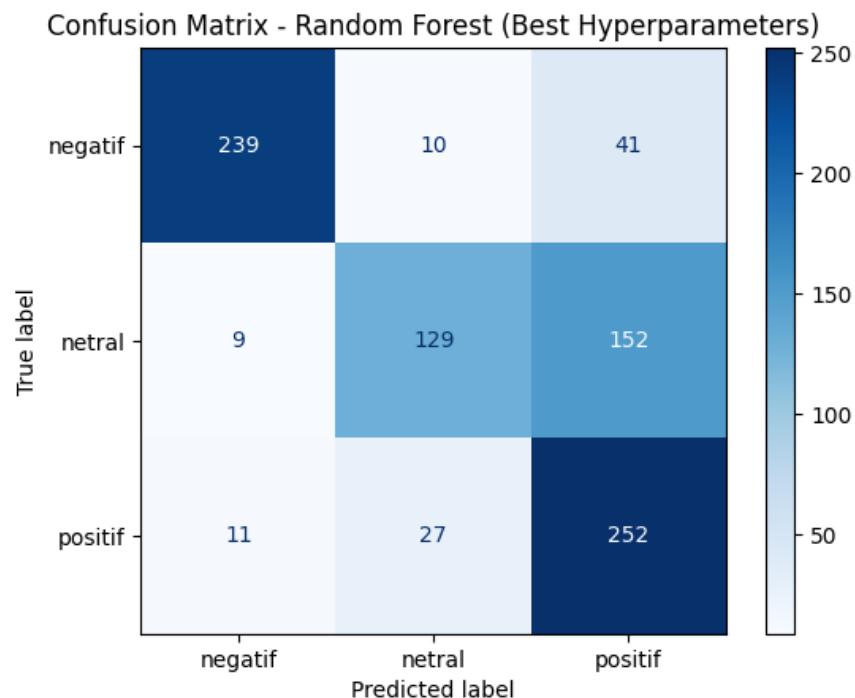
```
# Tuning Random Forest
rf_params = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}
rf_grid =
GridSearchCV(RandomForestClassifier(random_state=42),
rf_params, cv=5, scoring='accuracy')
rf_grid.fit(X_train, y_train)
rf_best = rf_grid.best_estimator_
rf_pred = rf_best.predict(X_test)
print("\n=== Random Forest ===")
print("Best Parameters:", rf_grid.best_params_)
print("Akurasi:", accuracy_score(y_test, rf_pred))
print(classification_report(y_test, rf_pred))
```

```
=== Random Forest ===
Best Parameters: {'max_depth': None, 'min_samples_split': 5, 'n_estimators': 200}
Akurasi: 0.7126436781609196
```

	precision	recall	f1-score	support
negatif	0.92	0.82	0.87	290
netral	0.78	0.44	0.57	290
positif	0.57	0.87	0.69	290
accuracy			0.71	870
macro avg	0.76	0.71	0.71	870
weighted avg	0.76	0.71	0.71	870

Visualisasi confusion matrix:

```
# Visualisasi confusion matrix untuk Random Forest hasil
hyperparameter tuning
cm_rf_best = confusion_matrix(y_test, rf_pred,
labels=rf_best.classes_)
disp_rf_best =
ConfusionMatrixDisplay(confusion_matrix=cm_rf_best,
display_labels=rf_best.classes_)
plt.figure(figsize=(5,4))
disp_rf_best.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - Random Forest (Best
Hyperparameters)")
plt.show()
```



Visualisasi learning curve random forest:

```
from sklearn.model_selection import learning_curve

# Learning curve untuk model Random Forest (hasil tuning)

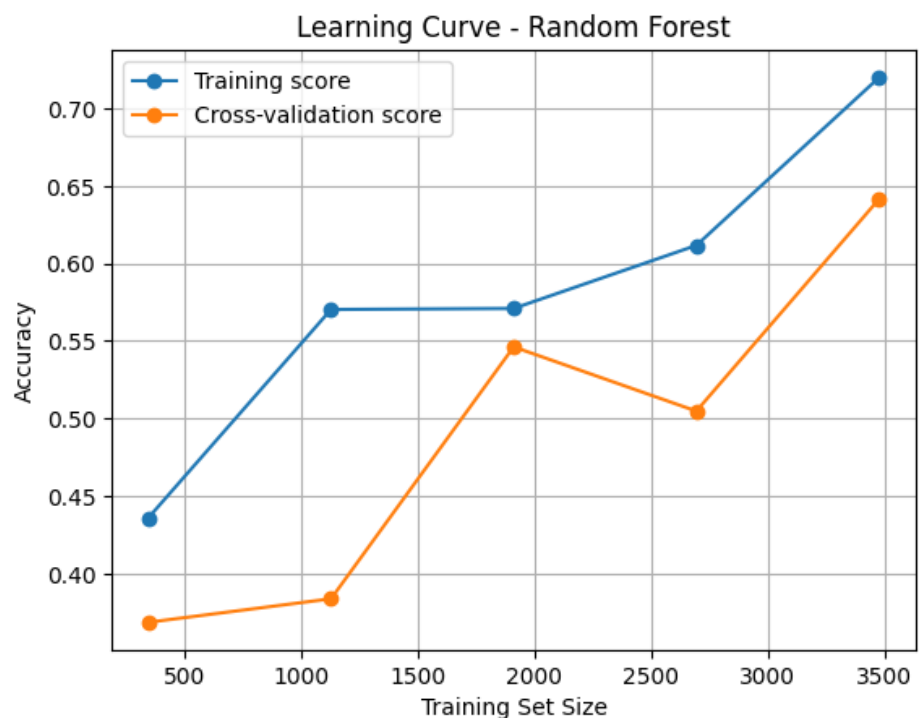
# Gunakan rf_best dari hasil GridSearchCV
train_sizes_rf, train_scores_rf, test_scores_rf =
learning_curve(
    rf_best, X, y, cv=5, scoring='accuracy', n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 5)
)
```

```

train_scores_mean_rf = train_scores_rf.mean(axis=1)
test_scores_mean_rf = test_scores_rf.mean(axis=1)

plt.plot(train_sizes_rf, train_scores_mean_rf, 'o-',
label="Training score")
plt.plot(train_sizes_rf, test_scores_mean_rf, 'o-',
label="Cross-validation score")
plt.xlabel("Training Set Size")
plt.ylabel("Accuracy")
plt.title("Learning Curve - Random Forest")
plt.legend(loc="best")
plt.grid()
plt.show()

```



6. Logistic Regression Model

Logistic Regression merupakan baseline yang solid untuk tugas klasifikasi biner seperti analisis sentimen. Meskipun sederhana, model ini sangat efisien dan efektif saat digabungkan dengan representasi TF-IDF. Logistic Regression bekerja baik dalam kasus teks karena model ini dapat memberikan probabilitas prediksi, yang bisa berguna dalam evaluasi lebih lanjut.

```

# == Model Logistic Regression ==
print("\n== Logistic Regression ==")
lr_model = LogisticRegression(max_iter=1000,
random_state=42)
lr_model.fit(X_train_imputed, y_train)
y_pred_lr = lr_model.predict(X_test_imputed)

```

```
print(classification_report(y_test, y_pred_lr))
print("Akurasi:", accuracy_score(y_test, y_pred_lr))
```

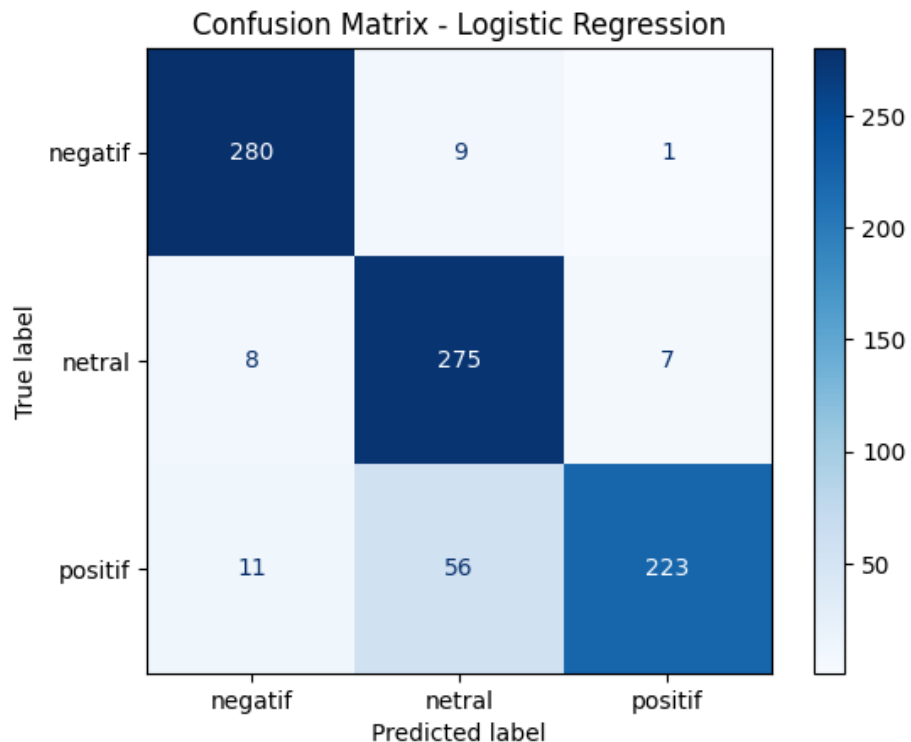
```
=== Logistic Regression ===
```

	precision	recall	f1-score	support
negatif	0.94	0.97	0.95	290
netral	0.81	0.95	0.87	290
positif	0.97	0.77	0.86	290
accuracy			0.89	870
macro avg	0.90	0.89	0.89	870
weighted avg	0.90	0.89	0.89	870

```
Akurasi: 0.8942528735632184
```

Visualisasi confusion matrix:

```
# Confusion matrix untuk model Logistic Regression
cm_lr = confusion_matrix(y_test, y_pred_lr,
labels=lr_model.classes_)
disp_lr = ConfusionMatrixDisplay(confusion_matrix=cm_lr,
display_labels=lr_model.classes_)
plt.figure(figsize=(5,4))
disp_lr.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```



Lakukan hyperparameter tuning:

```
# Tuning Logistic Regression
lr_params = {
    'C': [0.1, 1, 10],
    'solver': ['liblinear', 'lbfgs'],
    'penalty': ['l2']
}
lr_grid = GridSearchCV(LogisticRegression(max_iter=1000),
lr_params, cv=5, scoring='accuracy')
lr_grid.fit(X_train_imputed, y_train)
lr_best = lr_grid.best_estimator_
lr_pred = lr_best.predict(X_test_imputed)
print("\n=== Logistic Regression ===")
print("Best Parameters:", lr_grid.best_params_)
print("Akurasi:", accuracy_score(y_test, lr_pred))
print(classification_report(y_test, lr_pred))
```

=== Logistic Regression ===

Best Parameters: {'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}

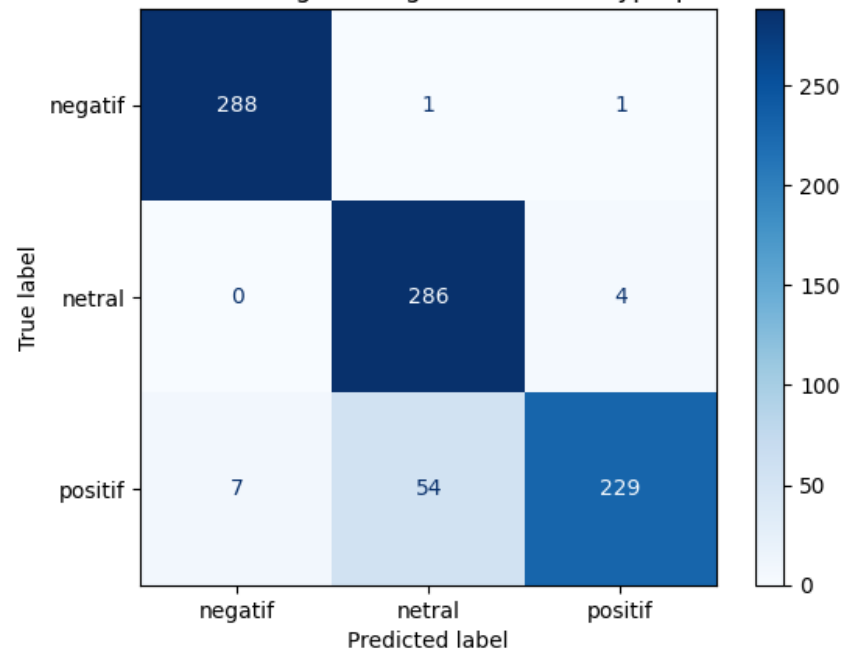
Akurasi: 0.9229885057471264

	precision	recall	f1-score	support
negatif	0.98	0.99	0.98	290
netral	0.84	0.99	0.91	290
positif	0.98	0.79	0.87	290
accuracy			0.92	870
macro avg	0.93	0.92	0.92	870
weighted avg	0.93	0.92	0.92	870

Visualisasi convusion matrix:

```
# Visualisasi confusion matrix untuk Logistic Regression
hasil hyperparameter tuning
cm_lr_best = confusion_matrix(y_test, lr_pred,
labels=lr_best.classes_)
disp_lr_best =
ConfusionMatrixDisplay(confusion_matrix=cm_lr_best,
display_labels=lr_best.classes_)
plt.figure(figsize=(5,4))
disp_lr_best.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix - Logistic Regression (Best
Hyperparameters)")
plt.show()
```

Confusion Matrix - Logistic Regression (Best Hyperparameters)



Visualisasi learning curve:

```
# Learning curve untuk model Logistic Regression (hasil
tuning)

train_sizes_lr, train_scores_lr, test_scores_lr =
learning_curve(
    lr_best, X_train_imputed, y_train, cv=5,
    scoring='accuracy', n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 5)
)

train_scores_mean_lr = train_scores_lr.mean(axis=1)
test_scores_mean_lr = test_scores_lr.mean(axis=1)

plt.plot(train_sizes_lr, train_scores_mean_lr, 'o-',
label="Training score")
plt.plot(train_sizes_lr, test_scores_mean_lr, 'o-',
label="Cross-validation score")
plt.xlabel("Training Set Size")
plt.ylabel("Accuracy")
plt.title("Learning Curve - Logistic Regression")
plt.legend(loc="best")
plt.grid()
plt.show()
```



V. LAMPIRAN

Source code lengkap: <https://github.com/gbennnn/sentiment-analysis-aperti-bumn>